МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ДЕРЖАВНИЙ ВИЩИЙ НАВЧАЛЬНИЙ ЗАКЛАД
«УЖГОРОДСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ»
ІНЖЕНЕРНО-ТЕХНІЧНИЙ ФАКУЛЬТЕТ
КАФЕДРА КОМП’ЮТЕРНИХ СИСТЕМ ТА МЕРЕЖ

**ЗВІТ**

до лабораторної роботи №5

з дисципліни «Програмування систем на кристалі»

Студента 4-го курсу
спеціальності 123 –
«Комп’ютерна інженерія»
Сеника Ярослава

м. Ужгород – 2026 р.

# ЛАБОРАТОРНА РОБОТА №5

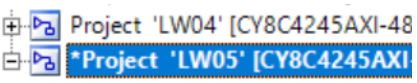
**Тема: Динамічна індикація, таймери, робота з логічним аналізатором у інтегрованому середовищі PSoC® CreatorTM**

**Мета:** Ознайомитися з принципами роботи динамічної індикації та налаштуванні переривань у мікроконтролерних системах на базі PSoC. Набути практичних навичок створення проєктів у інтегрованому середовищі PSoC® CreatorTM, налаштування апаратних компонентів для керування індикаторами через зсувні регістри та реалізації ефективного виведення даних. Також набути практичних навичок використання логічного аналізатора на прикладі Saleae Logic.

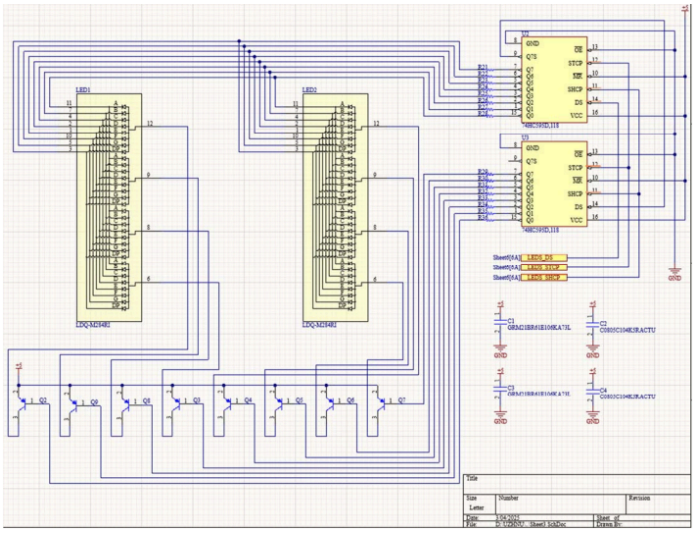
## Хід роботи

**Завдання 1.**

1. Скопіюйте проєкт Лабораторної роботи No4.



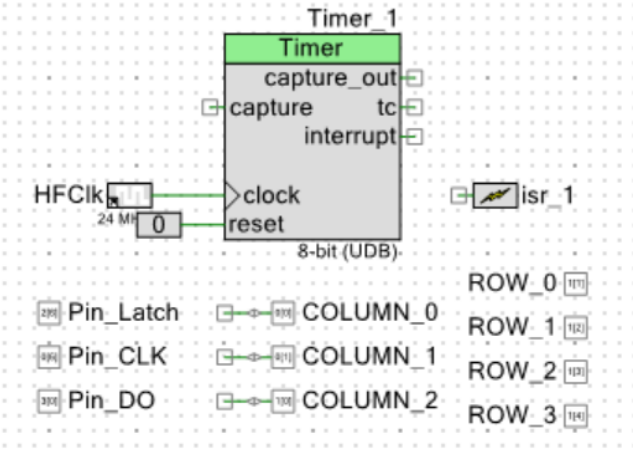

2. За посиланням можна ознайомитись з схемою Шилда для PSoC. На рис. зображено схему підключення семисегментних індикаторів до зсувних регістрів. Як видно на схемі, всі семисегментні символи паралельно підключені до першого зсувного регістра, а всі аноди до іншого. Тобто перші 8 біт керують символом який виводиться, а наступні 8 біт керують номером символа який потрібно увімкнути.



3. Написати код в main(), який відправляє дані почергово для кожного символа та запустити даний код, на індикаторах має бути послідовність значено від 7 до 0. Приклад показано на рис.

```
uint8_t data[8] = {7, 6, 5, 4, 3, 2, 1, 0};
for(;;)
{
    for(int i = 0; i <= 8; i++)
    {
        FourDigit74HC595_sendOneDigit(i, data[i], 0);
    }
```

4. Перейдіть на вкладку TopDesighn.cysch. Відредагуйте проєкт видаливши компоненти Лабораторної роботи No1 та No2, а саме Button, LED_R, LED_B, LED_G та SW_Tx_UART. З каталогу Cypress додайте на схему таймер Timer [v2.80] (підкаталог Digital, папка Functions) та з підкаталогу System, компоненту Interrupt [v1.70].

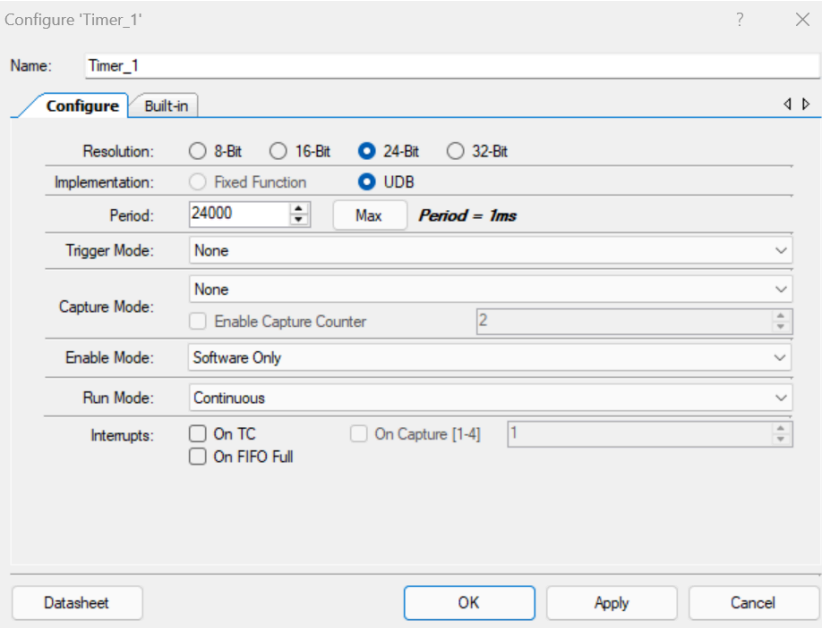



Виконуємо налаштування таймеру як зображено на рис, а саме:

Name: Timer;

Configure:

- o Resolution: 24 bit;
- o Implementation: UDB;
- o Period: 24000 (1 ms);
- o Trigger Mode: None;
- o Capture Mode: None;
- o Enable Mode: Software Only;
- o Run Mode: Continuous;
- o Interrups: —.




Компоненту Interrupt [v1.71] лише переіменовуємо Timer_Int.

5. Написати функцію обробника переривань.

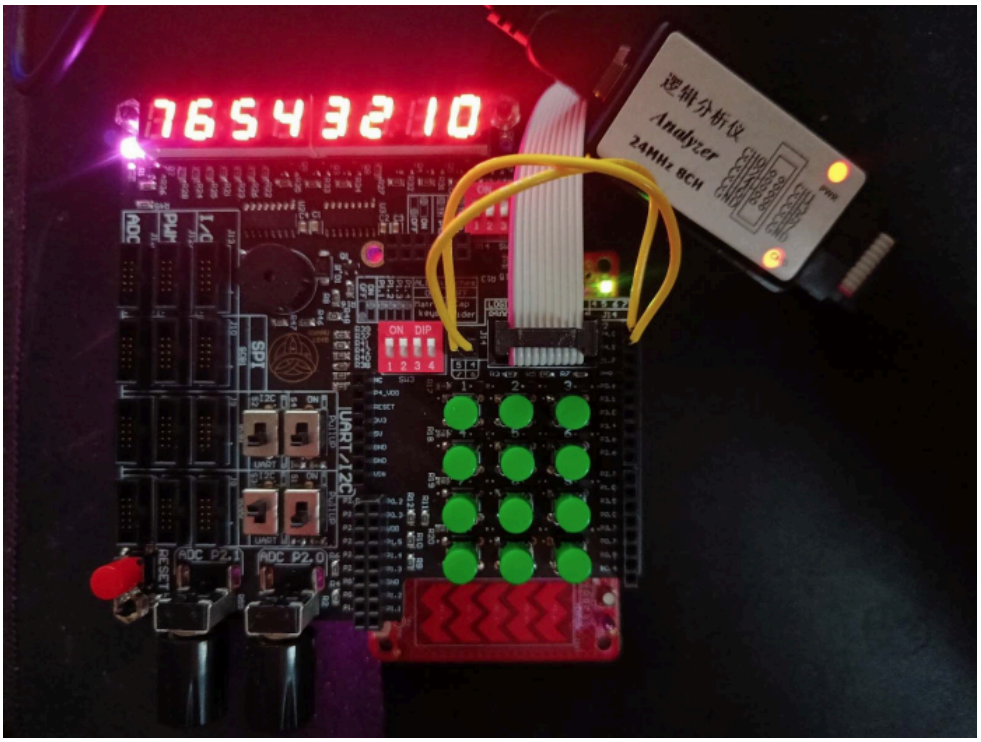
```
static uint8_t led_counter = 0;
CY_ISR(Timer_Int_Handler2)
{
    uint8_t data[8] = {7, 6, 5, 4, 3, 2, 1, 0};
    FourDigit74HC595_sendOneDigit(led_counter, data[led_counter], 0);
    led_counter++;
    if(led_counter > 7) led_counter = 0;
}
```

В main() запустити таймер та переривання.

```
Timer_Start();
Timer_Int_StartEx(Timer_Int_Handler2);
```

Запустивши даний код, на індикаторах послідовність значень від 7 до 0.

6. Провести експерименти з періодом таймера. Змінити його значення на 10 мс, або на 100 мс, проаналізувати отримані результати.




Завдання для самостійного виконання:

1. Додати можливість вводу паролю на семисегментний дисплей, після натиску на кнопку «#». Перевірити введений пароль з тим, який Ви оголосите самостійно в коді, якщо пароль вірний, виконати якусь дію на семисегментних індикаторах (може бути мигання або на ваш розсуд інша індикація). Також якщо пароль не вірний, потрібно сповістити користувача іншим паттерном мигання на семисегментних індикаторах.

2. Виконати циклічний вивід певного набору символів на семисегментних індикаторах за варіантами:

| Варіант | Зсув |
|---|---|
| 2 | Зліва – направо |

3. На перших чотирьох індикаторах реалізувати відлік 5 хвилин.

Лістинг коду:

```
/* ========================================
 *
 * Copyright YOUR COMPANY, THE YEAR
 * All Rights Reserved
 * UNPUBLISHED, LICENSED SOFTWARE.
 *
 * ========================================
```

```
*/
#include "project.h"

/*
==================================================================
======= *
 * НАЛАШТУВАННЯ ПАРОЛЮ
 *
==================================================================
======= */
#define PASS_LENGTH 7
const uint8 CORRECT_PASS[PASS_LENGTH] = {7, 3, 5, 5, 6, 0, 8}; //
пароль
uint8 entered_pass[8]; // Буфер для вводу з клавіатури
uint8 current_pass_len = 0;

/*
==================================================================
======= *
 * БУФЕР ДИСПЛЕЯ ТА СИМВОЛИ
 *
==================================================================
======= */
// Масив з 8 байтів, який постійно виводиться на дисплей через
переривання
volatile uint8 display_buffer[8] = {0xFF, 0xFF, 0xFF, 0xFF, 0xFF,
0xFF, 0xFF, 0xFF};

/* Функція для перетворення символів/букв у двійковий код
7-сегментного індикатора */
uint8 charToSeg(char c) {
    switch(c) {
        case 'A': return 0x88;
        case 'B': return 0x83; // b
        case 'C': return 0xC6;
        case 'D': return 0xA1; // d
        case 'E': return 0x86;
        case 'F': return 0x8E;
        case 'G': return 0x90; // виглядає як 9
        case 'H': return 0x89;
        case 'I': return 0xF9; // 1
        case 'L': return 0xC7;
        case 'M': return 0xC8; // специфічний символ, схожий на M
(або велику n)
        case 'N': return 0xAB; // n
        case 'O': return 0xC0; // 0
        case 'P': return 0x8C;
        case 'R': return 0xAF; // r
        case 'S': return 0x92; // 5
        case 'T': return 0x87; // t
        case '0': return 0xC0;
        case '1': return 0xF9;
        case '2': return 0xA4;
```

```
        case '3': return 0xB0;
        case '4': return 0x99;
        case '5': return 0x92;
        case '6': return 0x82;
        case '7': return 0xF8;
        case '8': return 0x80;
        case '9': return 0x90;
        case '-': return 0xBF;
        case ' ': return 0xFF; // порожньо
        default:  return 0xFF;
    }
}

/* Виведення текстового рядка на дисплей */
void printString(const char* str) {
    uint8 i = 0;
    while(i < 8) {
        if (str[i] == '\0') break;
        display_buffer[i] = charToSeg(str[i]);
        i++;
    }
    // Очищаємо залишок дисплея, якщо рядок коротший за 8 символів
    while(i < 8) {
        display_buffer[i] = 0xFF;
        i++;
    }
}

/* Оновлення дисплея при наборі паролю (вирівнювання по лівому
краю) */
void updatePassDisplay() {
    uint8 i;
    for(i = 0; i < 8; i++) {
        if(i < current_pass_len) {
            display_buffer[i] = charToSeg('0' + entered_pass[i]);
        } else {
            display_buffer[i] = 0xFF; // Пусті сегменти
        }
    }
}

/*
=================================================================
======= *
 * АПАРАТНА ЧАСТИНА (Зсувний регістр і Таймер)
 *
=================================================================
======= */
static void FourDigit74HC595_sendData(uint8 data) {
    uint8 i;
    for (i = 0; i < 8; i++) {
        if (data & (0x80 >> i)) Pin_DO_Write(1);
        else Pin_DO_Write(0);
```

```
        Pin_CLK_Write(1);
        Pin_CLK_Write(0);
    }
}

static uint8 led_counter = 0;

CY_ISR(Timer_Int_Handler2)
{
    // Щоб уникнути "накладання" символів (ghosting), спочатку
гасимо все
    FourDigit74HC595_sendData(0xFF);
    FourDigit74HC595_sendData(0xFF);
    Pin_Latch_Write(1);
    Pin_Latch_Write(0);

    // Виводимо поточний символ з буфера
    FourDigit74HC595_sendData(0xFF & ~(1 << led_counter)); //
Вибір позиції
    FourDigit74HC595_sendData(display_buffer[led_counter]); //
Дані символу

    Pin_Latch_Write(1);
    Pin_Latch_Write(0);

    led_counter++;
    if(led_counter > 7) led_counter = 0;
}

/*
==================================================================
======= *
 * ЛОГІКА МАТРИЧНОЇ КЛАВІАТУРИ
 *
==================================================================
======= */
static void (*COLUMN_x_SetDriveMode[3])(uint8 mode) = {
COLUMN_0_SetDriveMode, COLUMN_1_SetDriveMode,
COLUMN_2_SetDriveMode };
static void (*COLUMN_x_Write[3])(uint8 value) = { COLUMN_0_Write,
COLUMN_1_Write, COLUMN_2_Write };
static uint8 (*ROW_x_Read[4])(void) = { ROW_0_Read, ROW_1_Read,
ROW_2_Read, ROW_3_Read };

static uint8 keys[4][3] = { {1, 1, 1}, {1, 1, 1}, {1, 1, 1}, {1,
1, 1} };
static const uint8 button_ids[4][3] = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9},
    {10, 0, 11} // 10 = '*', 11 = '#'
};
```

```
static void initMatrix(void) {
    uint8 i;
    for(i = 0; i < 3; i++)
COLUMN_x_SetDriveMode[i](COLUMN_0_DM_DIG_HIZ);
}

static void readMatrix(void) {
    uint8 c, r;
    for(c = 0; c < 3; c++) {
        COLUMN_x_SetDriveMode[c](COLUMN_0_DM_STRONG);
        COLUMN_x_Write[c](0);
        for(r = 0; r < 4; r++) keys[r][c] = ROW_x_Read[r]();
        COLUMN_x_SetDriveMode[c](COLUMN_0_DM_DIG_HIZ);
    }
}

/*
=================================================================
======= *
 * АНІМАЦІЇ ДЛЯ УСПІХУ ТА НЕВДАЧІ
 *
=================================================================
======= */
void sequenceSuccess() {
    int16 total_sec;

    // 1. вивід на екран
    printString(" ACCEPT "); CyDelay(1000);
    printString("        "); CyDelay(200);
    printString("  BOMB  "); CyDelay(1000);
    printString("        "); CyDelay(200);
    printString("  HAS   "); CyDelay(1000);
    printString("        "); CyDelay(200);
    printString("  BEEN  "); CyDelay(1000);
    printString("        "); CyDelay(200);
    printString("PLANTED "); CyDelay(1000);
    printString("        "); CyDelay(200);

    // 6. Таймер
    for(total_sec = 10; total_sec >= 0; total_sec--) {
        uint8 min = total_sec / 60;
        uint8 sec = total_sec % 60;

        display_buffer[0] = 0xFF;
        display_buffer[1] = 0xFF;
        // Хвилини
        display_buffer[2] = charToSeg('0' + (min / 10));
        // Додаємо крапку до другої цифри хвилин (логічне АНД з
0x7F вмикає крапку)
        display_buffer[3] = charToSeg('0' + (min % 10)) & 0x7F;
        // Секунди
        display_buffer[4] = charToSeg('0' + (sec / 10));
```

```
        display_buffer[5] = charToSeg('0' + (sec % 10));
        display_buffer[6] = 0xFF;
        display_buffer[7] = 0xFF;

        CyDelay(1000); // Чекаємо рівно 1 секунду
    }

    printString("        "); // Очистити в кінці
}

void sequenceFail() {
    uint8 i, j, temp;

    // Пишемо по центру
    printString("FAILED  ");
    CyDelay(1000); // Показуємо 1 секунду, щоб користувач побачив

    // Робимо ефект плавання (біжучий рядок по колу)
    // 8 зсувів вправо повернуть текст на початкове місце
    for(i = 0; i < 8; i++) {
        temp = display_buffer[7]; // Зберігаємо останній символ
        // Зсуваємо всі символи вправо
        for(j = 7; j > 0; j--) {
            display_buffer[j] = display_buffer[j-1];
        }
        display_buffer[0] = temp; // Ставимо збережений символ на
початок

        CyDelay(300); // Швидкість плавання
    }
    CyDelay(500);
    printString("        "); // Очистити в кінці
}

/*
==================================================================
======= *
 * ОСНОВНИЙ ЦИКЛ ПРОГРАМИ
 *
==================================================================
======= */
int main(void) {
    uint8 r, c, i;
    uint8 last_pressed = 99;
    uint8 current_pressed;
    uint8 is_correct;

    CyGlobalIntEnable;

    Timer_Start();
    Timer_Int_StartEx(Timer_Int_Handler2);
    initMatrix();
```

```
    // Очищуємо дисплей при старті
    printString("        ");

    for(;;) {
        readMatrix();
        current_pressed = 99;

        for(r = 0; r < 4; r++) {
            for(c = 0; c < 3; c++) {
                if(keys[r][c] == 0) current_pressed =
button_ids[r][c];
            }
        }

        if(current_pressed != 99 && last_pressed == 99) {
            last_pressed = current_pressed;

            // Якщо натиснута кнопка '*' -> Скинути набір паролю
            if (current_pressed == 10) {
                current_pass_len = 0;
                updatePassDisplay();
            }
            // Якщо натиснута кнопка '#' -> Перевірити пароль
            else if (current_pressed == 11) {
                is_correct = 1;

                // Перевіряємо довжину і кожен символ
                if (current_pass_len != PASS_LENGTH) {
                    is_correct = 0;
                } else {
                    for(i = 0; i < PASS_LENGTH; i++) {
                        if(entered_pass[i] != CORRECT_PASS[i]) {
                            is_correct = 0;
                            break;
                        }
                    }
                }

                if (is_correct) {
                    sequenceSuccess();
                } else {
                    sequenceFail();
                }

                // Після анімації скидаємо буфер
                current_pass_len = 0;
                updatePassDisplay();
            }
            // Якщо натиснута цифра 0-9
            else {
                if (current_pass_len < 8) {
                    entered_pass[current_pass_len] =
current_pressed;
```

```
                    current_pass_len++;
                    updatePassDisplay();
                }
            }
            CyDelay(50); // Антибрязкіт
        }
        else if(current_pressed == 99 && last_pressed != 99) {
            last_pressed = 99;
            CyDelay(50); // Антибрязкіт
        }
    }
}
```

**Висновки:** Ознайомився з принципами роботи динамічної індикації та налаштуванні переривань у мікроконтролерних системах на базі PSoC. Набув практичних навичок створення проєктів у інтегрованому середовищі PSoC® CreatorTM, налаштував апаратні компоненти для керування індикаторами через зсувні регістри та реалізував виведення даних. Також набув практичних навичок використання логічного аналізатора на прикладі Saleae Logic.